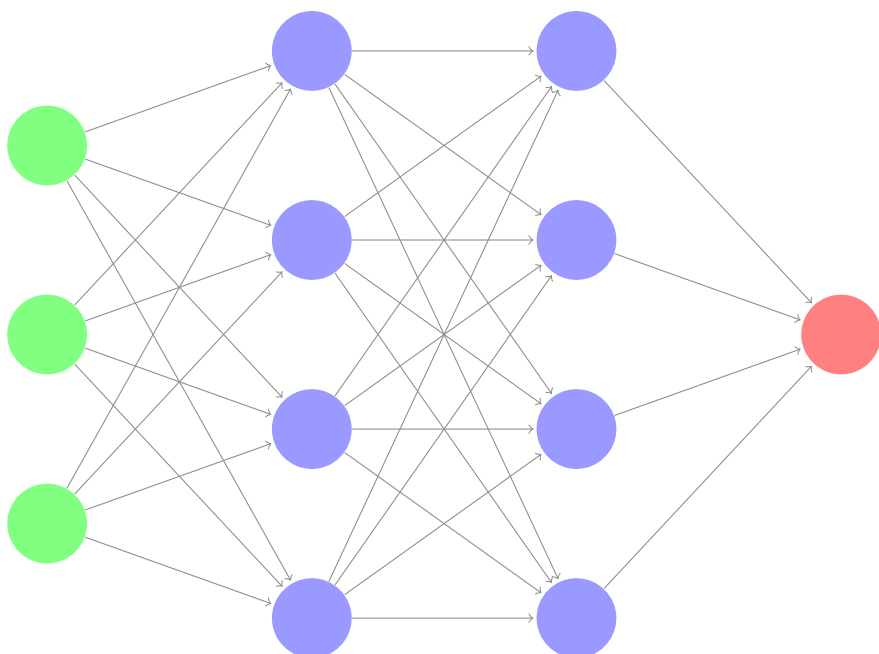


the math behind a neural network



hugo lageneste
hugo@olivia-ai.org

Contents

	Introduction	2
	1 Application example	2
	2 Neural network operation	2
A	Forward propagation	3
B	Back propagation	4
	1 Cost function	4
	2 Gradient descent	4
C	Complex example	5
	1 Forward propagation	5
	2 Back propagation	6

The math behind an artificial neural network

Introduction

1 Application example

Let's take an example to see how an ANN¹ works.

Obesity	Exercise	Smoking	Diabetic
1	0	0	1
0	1	0	0
0	0	1	0
1	1	0	1

Figure 1: Example of a set of persons with Obesity, Exercise, Smoking and Diabetic characteristics

In the precedent figure, the 1 stands for true and the 0 for false. We can observe that in this example, a person with diabetes is inevitably obese. What if we want a program which takes only in parameter those 4 examples and can predict with different examples if a person is diabetic or not?

We can model this program as an ANN with 3 input neurons which represent the Obesity, Exercise and Smoking columns and a single output neuron which represents the Diabetic column.

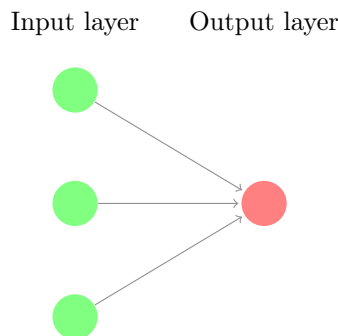


Figure 2: Diagram of the ANN to model the example

2 Neural network operation

The input values will be inserted in the input neurons and the network will operate as a “function” to say if the person is diabetic or not. Each liaison between each neuron is a value called weight and is unique. And each neuron holds a specific value, calculated with the previous neuron values and the weights. The whole goal of the neural network is to find the correct weights to make the final predictions right.

To make the weights right, we will proceed in different steps. First of all, the neural network will compute the value of the output layer's neurons named the prediction. Then, the network will make the difference between the prediction and the actual output and will adjust the weights to make the

¹Artificial Neural Network

prediction more accurate. This process will be repeated until the success rate is convenient.

After the construction of the neural network, we will be able to send values like 1, 1 and 1 (so it means that the person is obese, does exercise and smokes) and get an answer from the neural network which tells us that the person is diabetic.

A Forward propagation

As seen in the Introduction, each neuron holds a value contained between 0 and 1, calculated with the values of the previous neurons, the weights and a bias. We are going to see how these values are calculated.

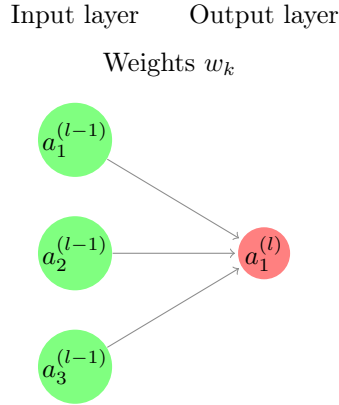


Figure 3: Diagram of a neural network, the exponent represents the index of the layer and the subscript the index of the neuron then a is the value held in the neuron.

Let's create a notation, $z_1^{(l)}$ which is the bias added to the dot product of weights w_k and values of the previous neurons $a_k^{(l-1)}$.

$$z_1^{(l)} = b_1^{(l)} + \sum_{k=1}^{n^{(l-1)}} a_k^{(l-1)} w_k$$

In order to keep the values in the neurons between 0 and 1, we are going to use the sigmoid function which is defined on $]0, 1[$, noted :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

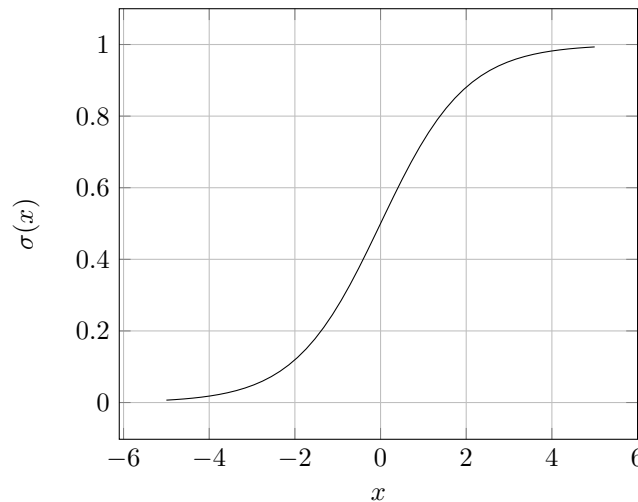


Figure 4: Graphical representation of the sigmoid function

Thus,

$$a_1^{(l)} = \sigma(z_1^{(l)})$$

$$a_1^{(l)} \in]0, 1[$$

This calculus can be too visualised with matrix

$$a_1^{(l)} = \sigma \left(\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ a_3^{(l-1)} \end{bmatrix} + \begin{bmatrix} b_1^{(l)} \end{bmatrix} \right)$$

B Back propagation

1 Cost function

Let's take a simple example of a one-input-layer-one-output-layer-neural-network

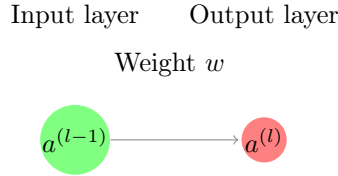


Figure 5: Diagram of the ANN to model the example of the cost function

Once again,

$$z^{(l)} = b + a^{(l-1)}w$$

$$a^{(l)} = \sigma(z^{(l)})$$

The goal is now to adjust the weights to make the prediction more accurate. Let's introduce the cost function which calculates the square difference between the prediction and the actual output y .

$$C_1(a^{(l)}, y) = (a^{(l)} - y)^2$$

The smaller the cost function is, the more accurate the predictions are, mathematically the goal is to minimize the cost function.

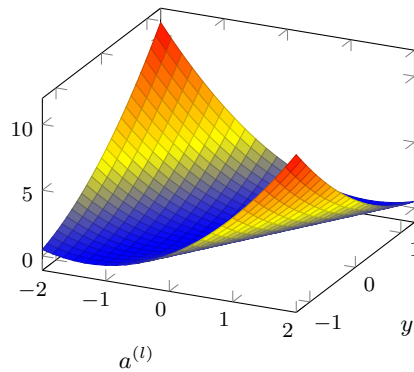


Figure 6: Cost function graphical representation of the neural network, figure 5

2 Gradient descent

Now we will need to understand how sensitive the cost function is to small changes to w_k because remember from 2, the goal is to adjust weights. Thus, we will determine the partial derivative of C with respect to w using the chain rule.

$$\frac{\partial C_1}{\partial w} = \frac{\partial C_1}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w}$$

Indeed,

$$\begin{aligned}\frac{\partial C_1}{\partial a^{(l)}} &= 2 \left(a^{(l)} - y \right) \\ \frac{\partial a^{(l)}}{\partial z^{(l)}} &= \frac{\partial \sigma(z^{(l)})}{\partial z^{(l)}} = \sigma'(z^{(l)}) \\ \frac{\partial z^{(l)}}{\partial w} &= \frac{\partial (b + a^{(l-1)}w)}{\partial w} = a^{(l-1)}\end{aligned}$$

All together, it gives us

$$\frac{\partial C_1}{\partial w} = 2 \left(a^{(l)} - y \right) \sigma'(z^{(l)}) a^{(l-1)}$$

We will use this formula to calculate the adjustments to make to the weights multiple times until the predictions are accurate.

$$w = w + \alpha \frac{\partial C_1}{\partial w}$$

C Complex example

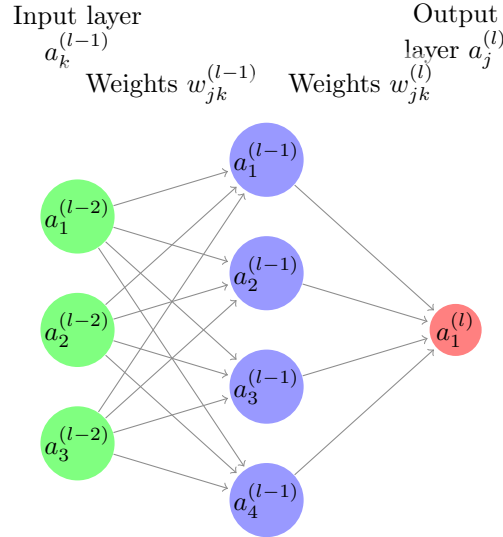


Figure 7: Diagram of a more complex ANN

We will model this problem with matrices.

1 Forward propagation

First of all we are going to create a matrix for each weights' layer

$$\begin{aligned}w^{(l-2)} &= \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix} \\ w^{(l-1)} &= \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \\ w_{41} \end{bmatrix}\end{aligned}$$

A layer is represented by a t by $k^{(l)}$ Matrix, where t is the number of training examples and k the number of nodes in a layer

$$a^{(l-2)} = \begin{bmatrix} a_1^{(l-2)} & a_2^{(l-2)} & a_3^{(l-2)} \end{bmatrix}$$

Then, to calculate the next layer's values we need to express z to compute a .

$$z^{(l)} = a^{(l-1)} \cdot w^{(l)} + b^{(l-1)}$$

$$a = \sigma(z)$$

2 Back propagation

Because we have now a hidden layer, we will need to express the partial derivative of C with respect to $w^{(l)}$ couched in terms of l .

The adjustments will take the following form

$$w^{(l)} = w^{(l)} + \alpha \frac{\partial C}{\partial w^{(l)}}$$

Here, α represents the learning rate.

Thus, we need to compute this derivative $\frac{\partial C}{\partial w^{(l)}}$. Using the chain rule,

$$\frac{\partial C}{\partial w^{(l)}} = \frac{\partial C}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial w^{(l)}}$$

First of all let's compute $\frac{\partial z^{(l)}}{\partial w^{(l)}}$

$$\frac{\partial z^{(l)}}{\partial w^{(l)}} = \frac{\partial}{\partial w^{(l)}} \left(a^{(l-1)} w^{(l)} + b^{(l-1)} \right) = a^{(l-1)}$$

Then, we'll compute $\frac{\partial C}{\partial z^{(l)}}$ couched in terms of $z^{(l+1)}$ in order to model backpropagation in programming.

$$\frac{\partial C}{\partial z^{(l)}} = \frac{\partial C}{\partial z^{(l+1)}} \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}}$$

$$\frac{\partial z^{(l+1)}}{\partial a^{(l)}} = \frac{\partial}{\partial a^{(l)}} \left(w^{(l+1)} a^{(l)} + b^{(l)} \right) = w^{(l+1)}$$

$$\frac{\partial a^{(l)}}{\partial z^{(l)}} = \frac{\partial}{\partial z^{(l)}} \sigma \left(z^{(l)} \right) = \sigma' \left(z^{(l)} \right)$$

Thus, after adjusting the terms to make dot products working we got

$$\frac{\partial C}{\partial z^{(l)}} = \left(w^{(l+1)^T} \cdot \frac{\partial C}{\partial z^{(l+1)}} \right) \times \sigma' \left(z^{(l)} \right)$$

And,

$$\frac{\partial C}{\partial w^{(l)}} = \left(w^{(l+1)^T} \cdot \frac{\partial C}{\partial z^{(l+1)}} \right) \times \sigma' \left(z^{(l)} \right) \times a^{(l-1)}$$

As you can see here, to compute the adjustments for the weights you need the derivative of C with respect to the next z so we will need to calculate the derivative of C with respect to z for the last layer so we never run get out of the range. Here L is the last layer

$$\frac{\partial C}{\partial z^{(L)}} = 2(a^{(L)} - y) \sigma' \left(z^{(L)} \right)$$

$$\frac{\partial C}{\partial z^{(L)}} = 2(a^{(L)} - y) \sigma \left(z^{(L)} \right) \left(1 - \sigma \left(z^{(L)} \right) \right)$$

$$\frac{\partial C}{\partial z^{(L)}} = 2(a^{(L)} - y) a^{(L)} \left(1 - a^{(L)} \right)$$

Here we have all the resources we need to build an artificial neural network.

Python code

Back propagation neural

network

Prof Happy AI, 2026

Chào bạn! Đoạn nội dung bạn cung cấp mô tả quá trình **Forward Propagation** (Lan truyền tiến) cơ bản trong một mạng nơ-ron nhân tạo.

Dưới đây là mã Python sử dụng thư viện **NumPy** để thực hiện các phép toán ma trận này một cách tối ưu.

1. Cấu trúc mã Python

Để tính giá trị của lớp hiện tại từ lớp trước đó, chúng ta thực hiện công thức:

$$a^{(l)} = \sigma(W \cdot a^{(l-1)} + b^{(l)})$$

Python

```
import numpy as np

def sigmoid(x):
    """Hàm kích hoạt Sigmoid đưa giá trị về khoảng (0, 1)"""
    return 1 / (1 + np.exp(-x))

def forward_propagation_step(a_prev, weights, bias):
    """
    Thực hiện tính toán cho một tầng (layer)
    a_prev: Vector giá trị của tầng trước (layer l-1)
    weights: Ma trận trọng số (w)
    bias: Độ lệch (b)
    """
    # 1. Tính tổng có trọng số: z = w * a + b
    z = np.dot(weights, a_prev) + bias
```

Tiếp nối phần Forward Propagation, đoạn nội dung này mô tả cốt lõi của việc học máy:

Backpropagation (Lan truyền ngược). Mục tiêu là tính toán xem lỗi (Cost) thay đổi như thế nào khi ta thay đổi trọng số (w), từ đó cập nhật w để giảm thiểu lỗi đó.

Dưới đây là mã Python thực hiện quá trình tính đạo hàm (gradient) và cập nhật trọng số theo đúng các công thức bạn đã cung cấp.

1. Mã Python thực hiện Backpropagation

Chúng ta sẽ sử dụng đạo hàm của hàm Sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Python

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    # Đạo hàm của hàm sigmoid: sigma(z) * (1 - sigma(z))
    s = sigmoid(x)
    return s * (1 - s)

# --- Khởi tạo giá trị giả lập ---
a_prev = 0.8 # Giá trị nơ-ron lớp trước a(l-1)
y = 1.0      # Giá trị thực tế mong muốn (label)
w = 0.5      # Trọng số hiện tại
b = 0.1      # Bias
alpha = 0.1  # Tốc độ học (Learning rate)

# --- 1. Forward Pass (Để lấy giá trị dự đoán) ---
z = b + a_prev * w
a_curr = sigmoid(z)

# --- 2. Backpropagation (Tính các đạo hàm riêng) ---
# dC/da = 2 * (a - y)
dC_da = 2 * (a_curr - y)

# da/dz = sigma'(z)
da_dz = sigmoid_derivative(z)

# dz/dw = a_prev
dz_dw = a_prev

# Chain Rule: dC/dw = dC/da * da/dz * dz/dw
dC_dw = dC_da * da_dz * dz_dw

# --- 3. Cập nhật trọng số (Gradient Descent) ---
# Lưu ý: Trong công thức của bạn là w = w + alpha * dC/dw
```

```
# Tuy nhiên, để giảm Cost, chúng ta thường dùng dấu trừ: w = w - alpha * dC/dw
w_new = w - alpha * dC_dw

print(f"Dự đoán ban đầu (a): {a_curr:.4f}")
print(f"Lỗi (Cost): {(a_curr - y)**2:.4f}")
print(f"Đạo hàm dC/dw: {dC_dw:.4f}")
print(f"Trọng số mới sau khi cập nhật: {w_new:.4f}")
```

2. Giải thích các thành phần toán học

Hàm Cost (Hàm mất mát)

Công thức $C = (a^{(l)} - y)^2$ cho chúng ta một đồ thị hình parabol. Khi giá trị dự đoán a càng gần giá trị thực y , đáy của parabol càng thấp (lỗi bằng 0).

Quy tắc chuỗi (Chain Rule)

Để biết w cần thay đổi bao nhiêu, chúng ta "đi ngược" từ đầu ra về đầu vào:

1. **Lỗi thay đổi thế nào theo đầu ra?** $\frac{\partial C}{\partial a}$
2. **Đầu ra thay đổi thế nào theo tổng đầu vào?** $\frac{\partial a}{\partial z}$
3. **Tổng đầu vào thay đổi thế nào theo trọng số?** $\frac{\partial z}{\partial w}$

Nhân tất cả lại, ta có "độ dốc" (gradient) của hàm lỗi theo trọng số w .

Cập nhật trọng số (Gradient Descent)

Công thức trong tài liệu của bạn là $w = w + \alpha \frac{\partial C}{\partial w}$.

- **Lưu ý quan trọng:** Thông thường trong AI, chúng ta dùng dấu **trừ** ($w = w - \alpha \nabla C$) vì đạo hàm chỉ hướng đi **lên**, mà chúng ta thì muốn đi **xuống** đáy của hàm lỗi. Nếu tài liệu của bạn dùng dấu cộng, có thể họ đang định nghĩa hướng điều chỉnh ngược lại hoặc α mang giá trị âm.
-

3. Tại sao cần Backpropagation?

Nếu không có nó, máy tính sẽ không biết phải tăng hay giảm w bao nhiêu để dự đoán chính xác hơn. Backpropagation cung cấp một con số cụ thể và hướng đi chính xác để tối ưu hóa mạng nơ-ron qua từng vòng lặp (epoch).

Chào bạn, đây là bước tiến lớn từ một nơ-ron đơn lẻ sang một **Mạng Nơ-ron Đa tầng (Multi-layer Neural Network)** sử dụng đại số tuyến tính (ma trận).

Để giải quyết bài toán này, chúng ta sẽ sử dụng **NumPy** để xử lý các phép nhân ma trận. Công thức quan trọng nhất ở đây là việc sử dụng Transpose (chuyển vị) ma trận w^T để lan truyền lỗi ngược về các lớp phía trước.

1. Mã Python cho Mạng Nơ-ron phức hợp

Python

```
import numpy as np

class NeuralNetwork:
    def __init__(self, layers, learning_rate=0.1):
        self.layers = layers # Ví dụ: [3, 4, 1] - 3 input, 4 hidden, 1 output
        self.alpha = learning_rate

        # Khởi tạo Weights và Biases ngẫu nhiên
        self.weights = [np.random.randn(self.layers[i], self.layers[i+1])
                        for i in range(len(self.layers)-1)]
        self.biases = [np.random.randn(1, self.layers[i+1])
                       for i in range(len(self.layers)-1)]

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, a):
        # Đạo hàm dựa trên giá trị a đã qua activation: a * (1 - a)
        return a * (1 - a)

    def forward(self, x):
        # Lưu trữ các giá trị a và z để dùng cho backprop
        self.a = [x]
        self.z = []

        for i in range(len(self.weights)):
            z = np.dot(self.a[-1], self.weights[i]) + self.biases[i]
            a = self.sigmoid(z)
            self.z.append(z)
            self.a.append(a)
        return self.a[-1]

    def backward(self, x, y):
        m = x.shape[0]
        # 1. Tính lỗi tại lớp cuối cùng (L)
        # dC/dz(L) = 2(a(L) - y) * a(L)(1 - a(L))
        delta = 2 * (self.a[-1] - y) * self.sigmoid_derivative(self.a[-1])
```

```

# Danh sách lưu trữ các đạo hàm để cập nhật
dW = []
db = []

# 2. Lan truyền ngược qua các lớp
for l in reversed(range(len(self.weights))):
    # dC/dw(l) = a(l-1).T * delta
    dW_l = np.dot(self.a[l].T, delta)
    db_l = np.sum(delta, axis=0, keepdims=True)

    dW.append(dW_l)
    db.append(db_l)

    # Tính delta cho lớp trước đó (l-1)
    # dC/dz(l-1) = (w(l) * delta) * sigma'(z(l-1))
    if l > 0:
        delta = np.dot(delta, self.weights[l].T) * self.sigmoid_derivative(self.a

# Đảo ngược danh sách vì ta tính từ cuối lên
dW.reverse()
db.reverse()

# 3. Cập nhật trọng số (Gradient Descent)
for i in range(len(self.weights)):
    # Lưu ý: w = w - alpha * dW (để giảm thiểu Cost)
    self.weights[i] -= self.alpha * dW[i]
    self.biases[i] -= self.alpha * db[i]

# --- Sử dụng thử nghiệm ---
X = np.array([[0.5, 0.2, 0.9]]) # 1 ví dụ với 3 tính năng
Y = np.array([[1.0]])           # Kết quả mong muốn

nn = NeuralNetwork([3, 4, 1]) # Input(3) -> Hidden(4) -> Output(1)

# Training thử 1000 lần
for i in range(1000):
    output = nn.forward(X)
    nn.backward(X, Y)
    if i % 200 == 0:
        cost = np.mean((output - Y)**2)
        print(f"Epoch {i}, Cost: {cost:.6f}")

print(f"Dự đoán cuối cùng: {nn.forward(X)}")

```

2. Giải thích các công thức Ma trận

Trong ví dụ phức tạp này, chúng ta sử dụng **Vectorization** (Vector hóa) để thay thế các phép tính riêng lẻ.

Forward Propagation

Công thức ma trận cho mỗi tầng:

$$z^{(l)} = a^{(l-1)} \cdot W^{(l)} + b^{(l-1)}$$

Trong đó $a^{(l-1)}$ là ma trận kích thước $(t \times k)$ (với t là số mẫu dữ liệu, k là số nơ-ron).

Backpropagation với Chain Rule

Điểm mấu chốt trong tài liệu của bạn là công thức tính $\frac{\partial C}{\partial z^{(l)}}$ dựa trên lớp kế tiếp:

$$\frac{\partial C}{\partial z^{(l)}} = (W^{(l+1)T} \cdot \frac{\partial C}{\partial z^{(l+1)}}) \odot \sigma'(z^{(l)})$$

- Dấu $\cdot$ là nhân ma trận (dot product).
- Dấu $\odot$ (trong code là `*`) là nhân từng phần tử tương ứng (Hadamard product).

3. Những điểm cần lưu ý khi lập trình

1. **Chuyển vị ma trận (W^T):** Rất quan trọng để khớp kích thước khi nhân ngược lỗi từ lớp có ít nơ-ron về lớp có nhiều nơ-ron hơn.
2. **Đạo hàm Sigmoid:** Trong mã, mình dùng `a * (1 - a)` vì $a = \sigma(z)$, giúp tính toán nhanh hơn việc gọi lại hàm `exp`.
3. **Tốc độ học (α):** Nếu bạn để α quá lớn, mạng sẽ bị mất ổn định; nếu quá nhỏ, nó sẽ học rất chậm.

Cấu trúc này chính là nền tảng của các thư viện lớn như PyTorch hay TensorFlow! Bạn có thể thêm nhiều tầng hơn vào danh sách `[3, 4, 8, 4, 1]` để thấy sức mạnh của Deep Learning.